COESIONE ITALIA 21-27 PIEMONTE
Cofinanziato dall'Unione europea
REGIONE PIEMONTE

Software Developer
Unità Formativa (UF): Containers, Microservizi - Serverless
Docente: Denis Maggiorotto
Titolo argomento: Introduzione a kubernetes

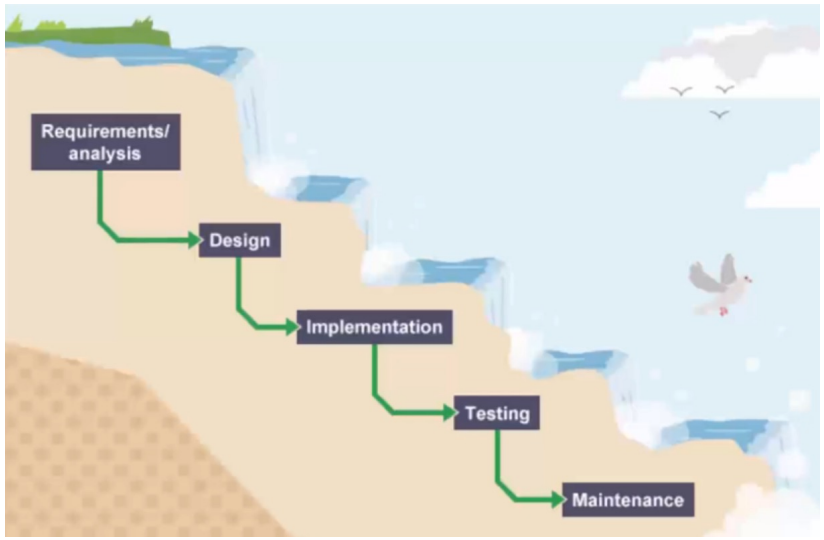
Intervento realizzato da ITS TECNOLOGIE DELL'INFORMAZIONE E DELLA COMUNICAZIONE

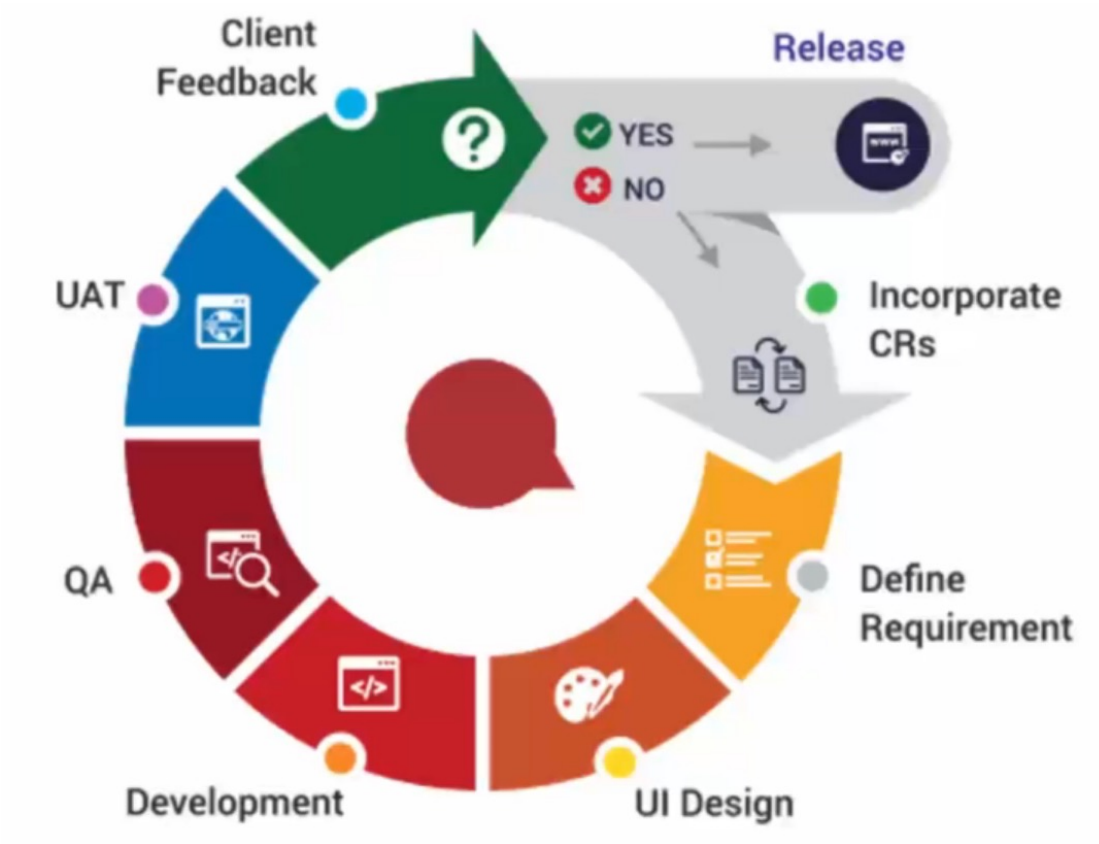
Kubernetes

L'orchestratore di container per eccellenza

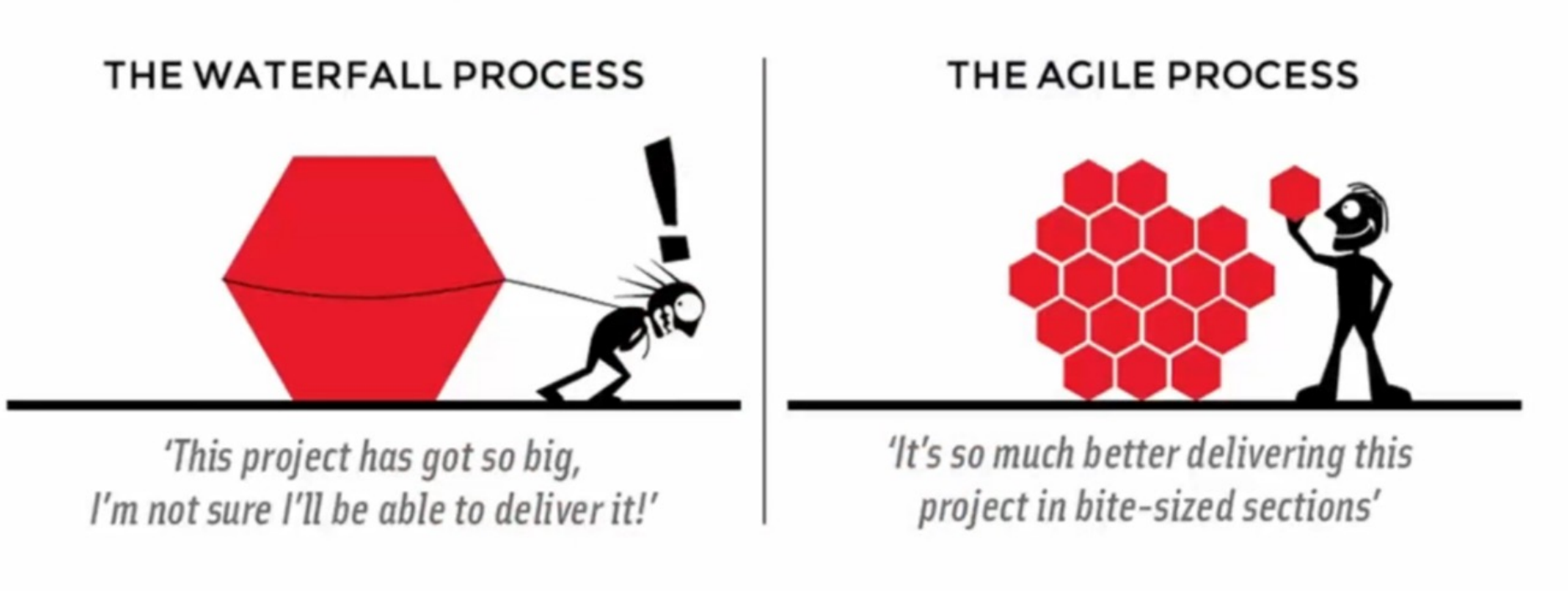
Dal monolite ai micro servizi

Lo sviluppo diventa agile

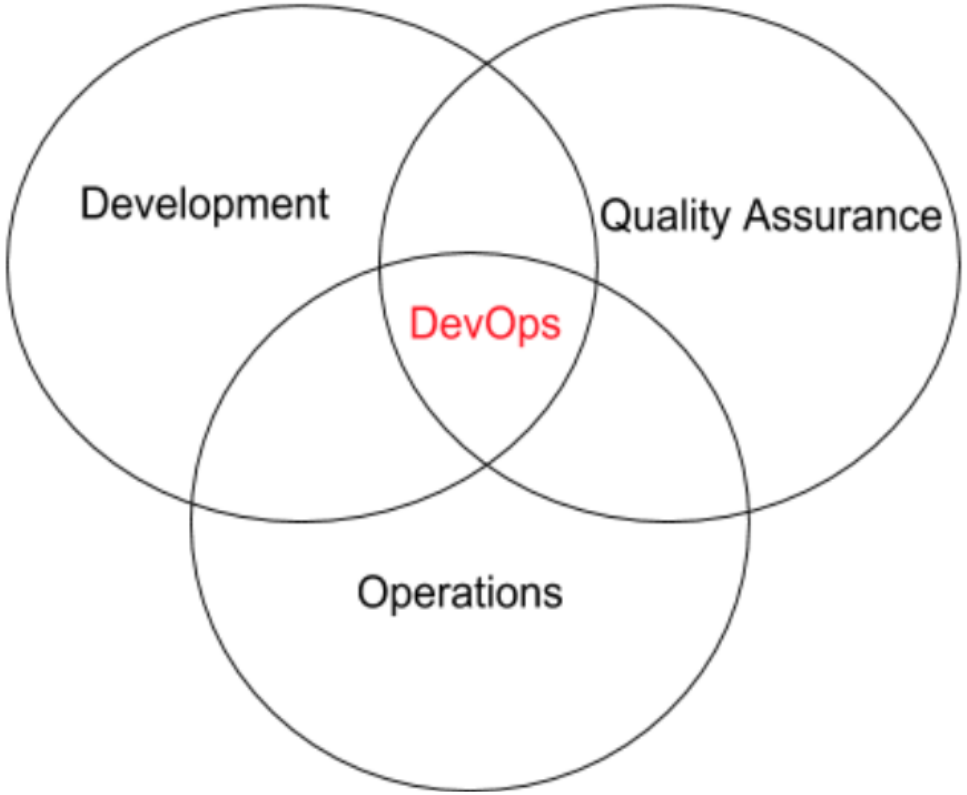




Waterfall Vs Agile



Il movimento DevOps



I principi del DevOps (CALMS)

Culture: there is a culture of shared responsibility

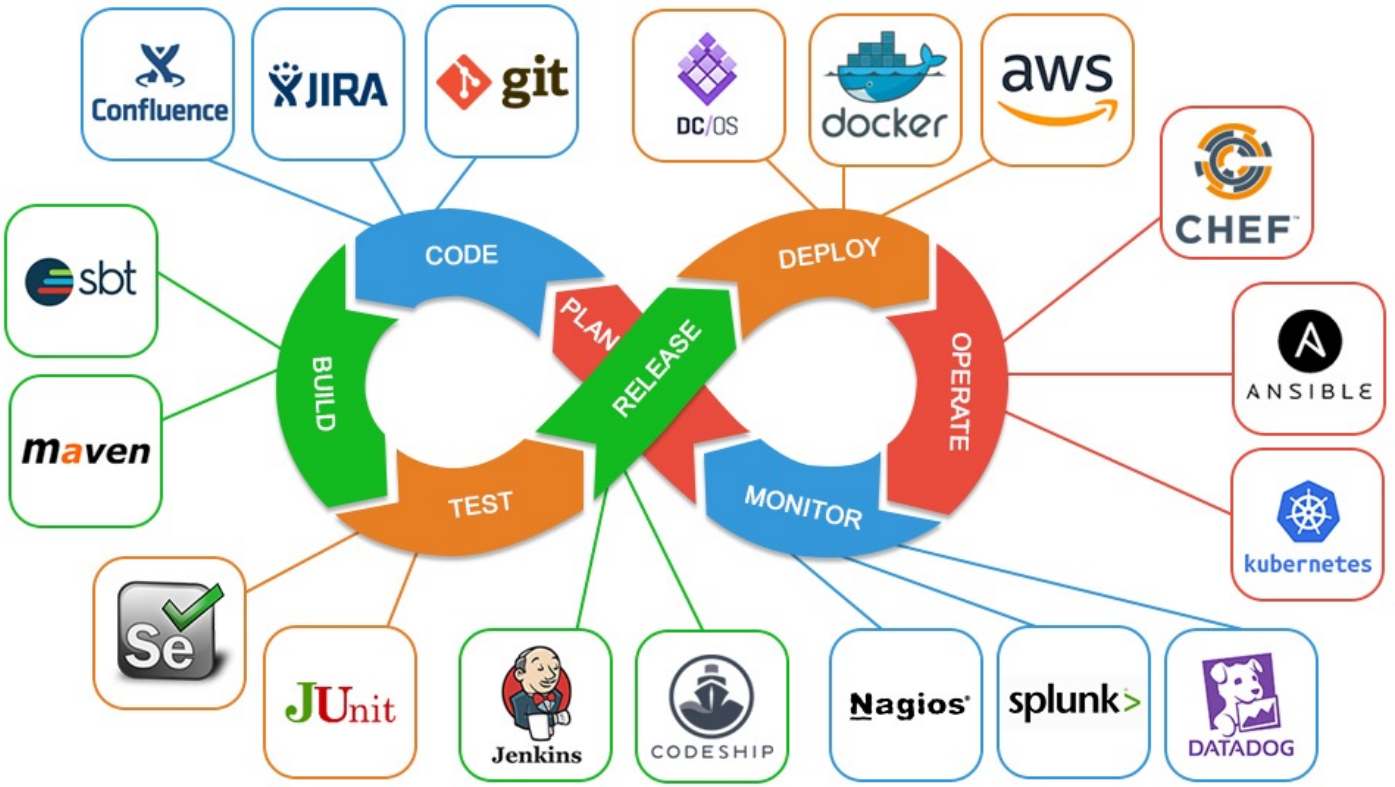
Automation: team members seek out ways to automate as many tasks as possible and are comfortable with the idea of continuous delivery

Lean: team members are able to visualize work in progress (WIP), limit batch sizes and manage queue lengths

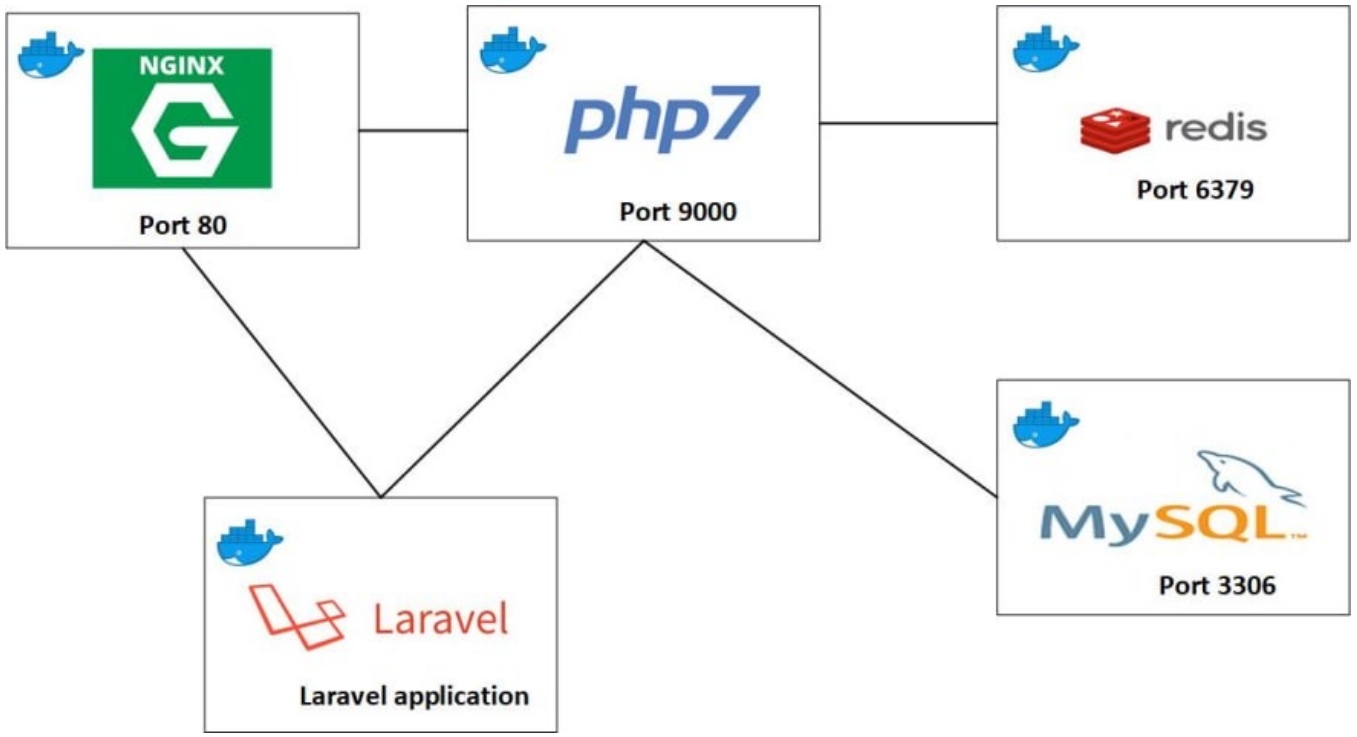
Measurement: data is collected on everything and there are mechanisms in place that provide visibility into all systems

Sharing: (a.k.a. Collaboration) there are user-friendly communication channels that encourage ongoing communication between development and operations

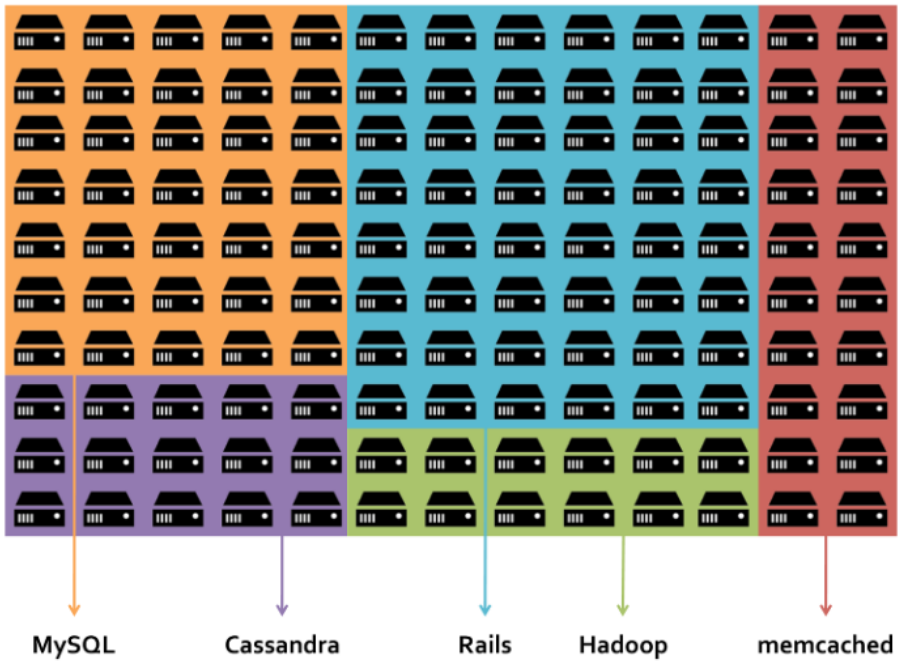
Gli strumenti del DevOps (solo una piccola parte)



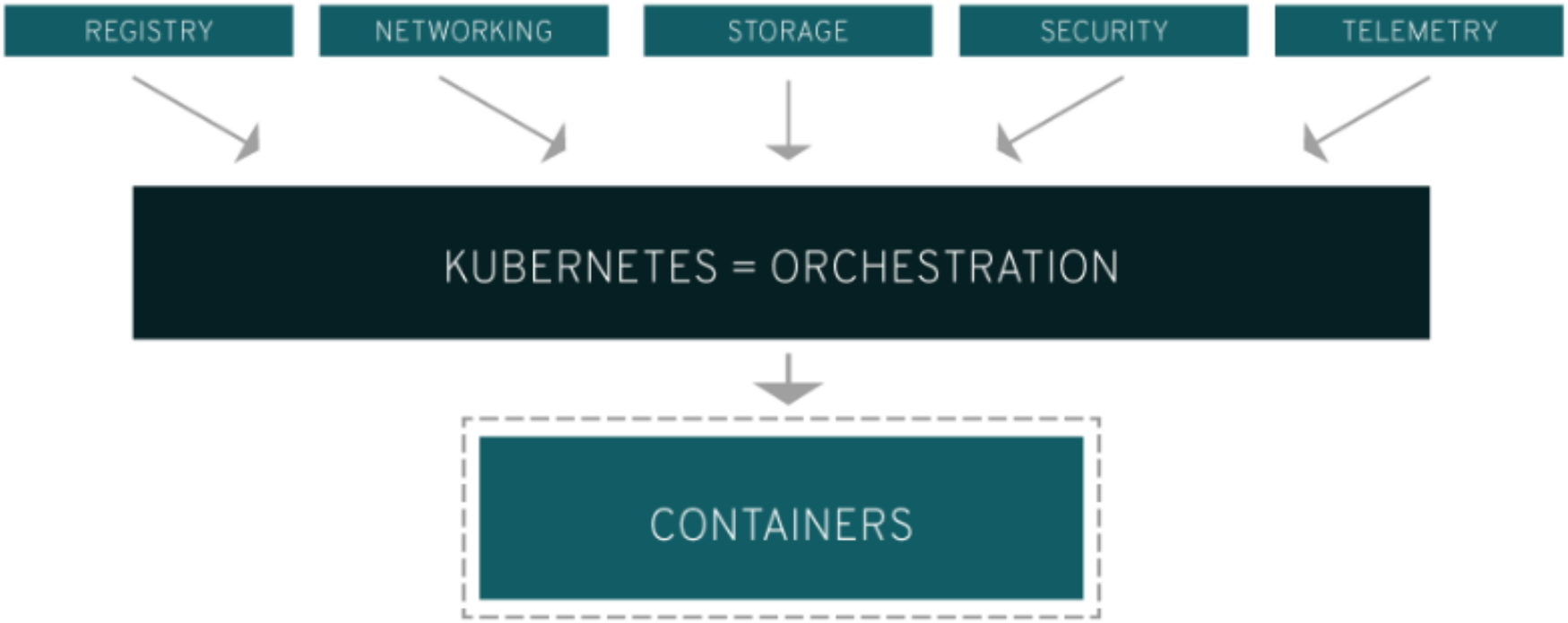
Le applicazioni diventano multi container (microservices)



Troppi container?



Perché Kubernetes

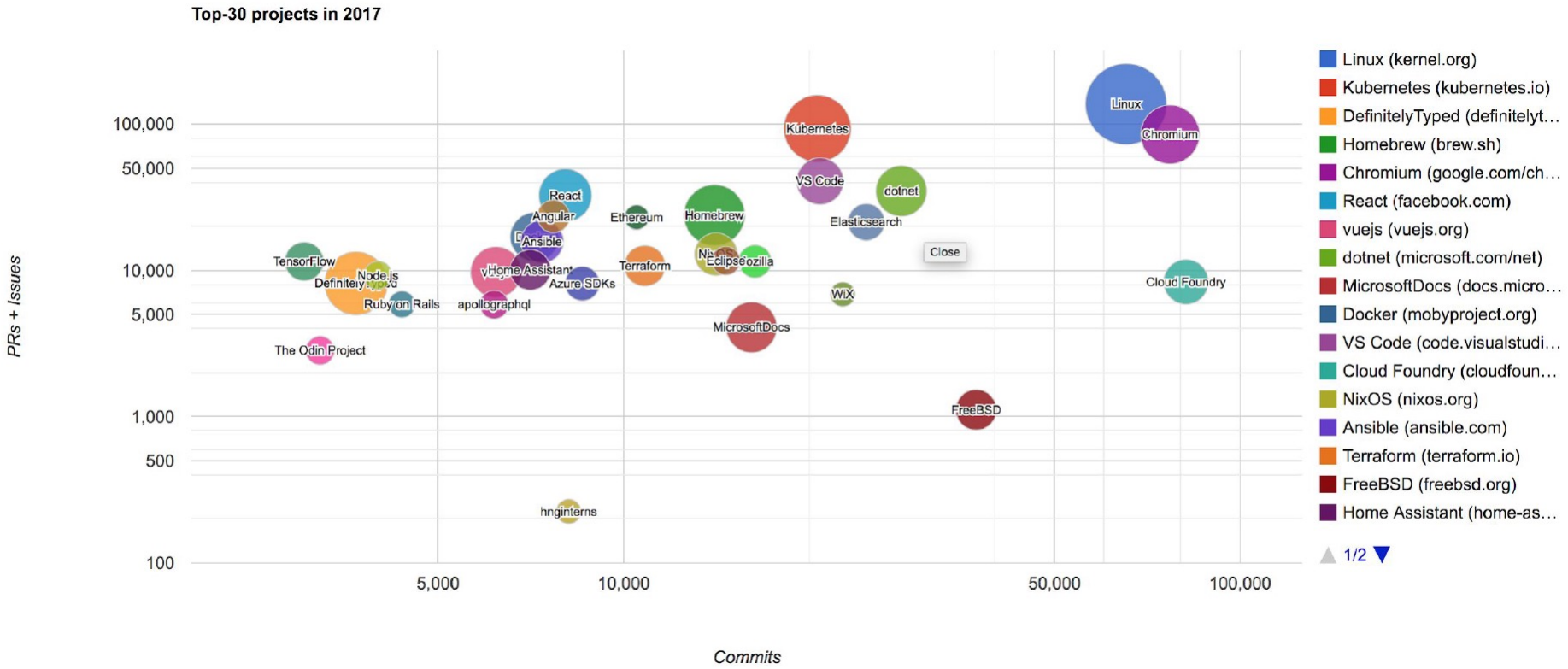


Cos’è Kubernetes

Kubernetes

Kubernetes is an open source container orchestration engine for automating deployment, scaling, and management of containerized applications.
The open source project is hosted by the Cloud Native Computing Foundation.

Kubernetes dal 2017 è il secondo progetto OO dopo Linux



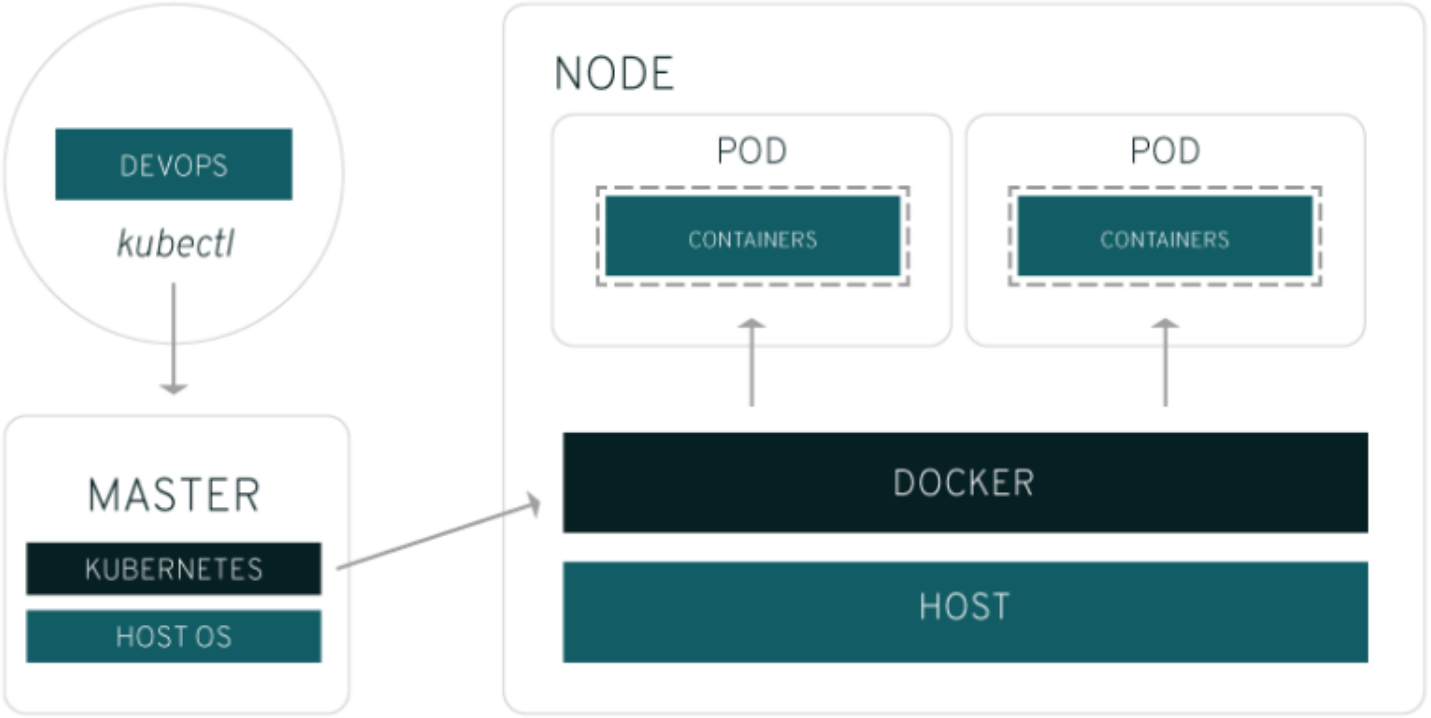
LAB

https://github.com/sunnyvale-academy/ITS-ICT_Containers

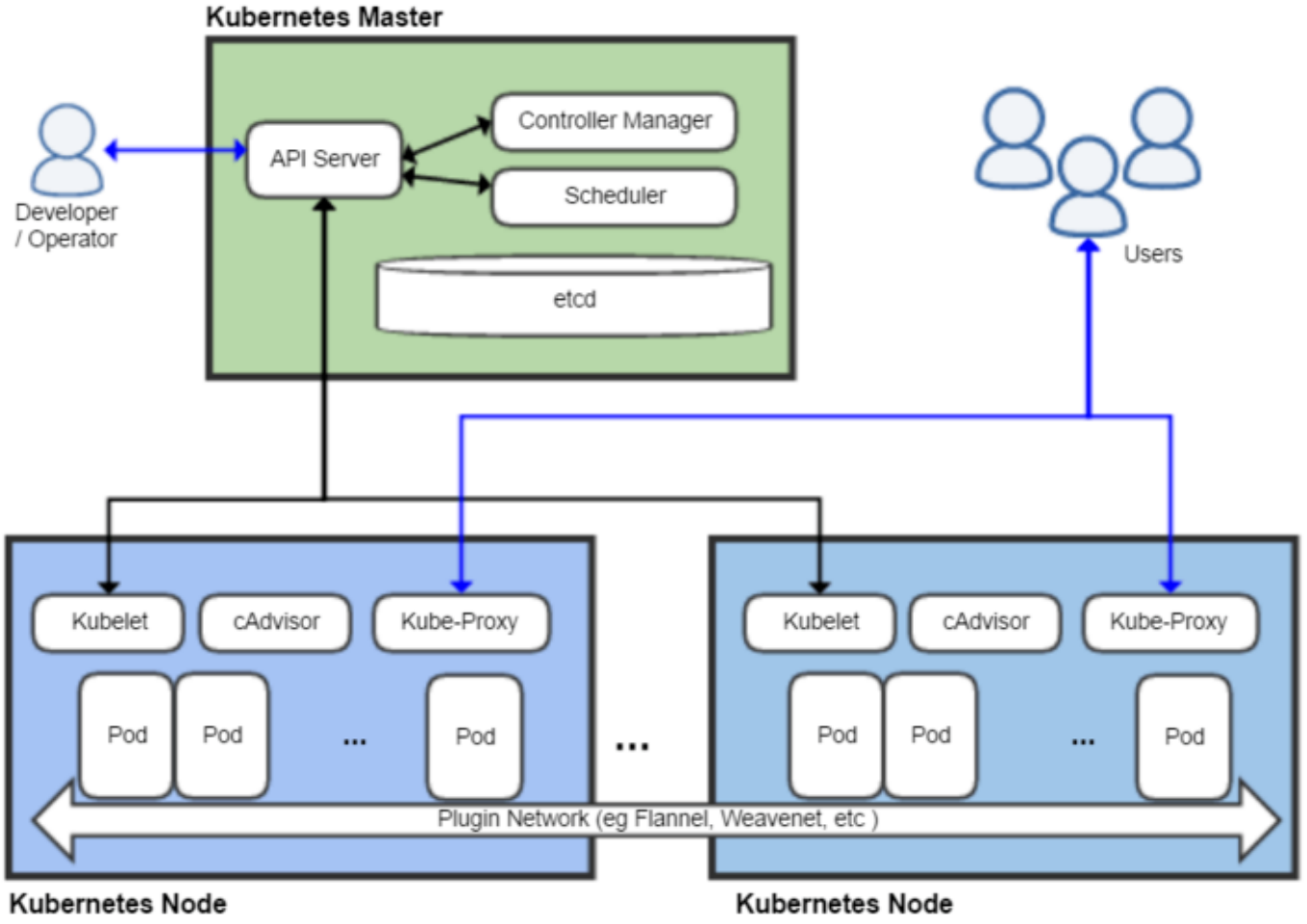
Lab 18 – Kubernetes

L’architettura di Kubernetes ed il Pod

L’architettura di Kubernetes



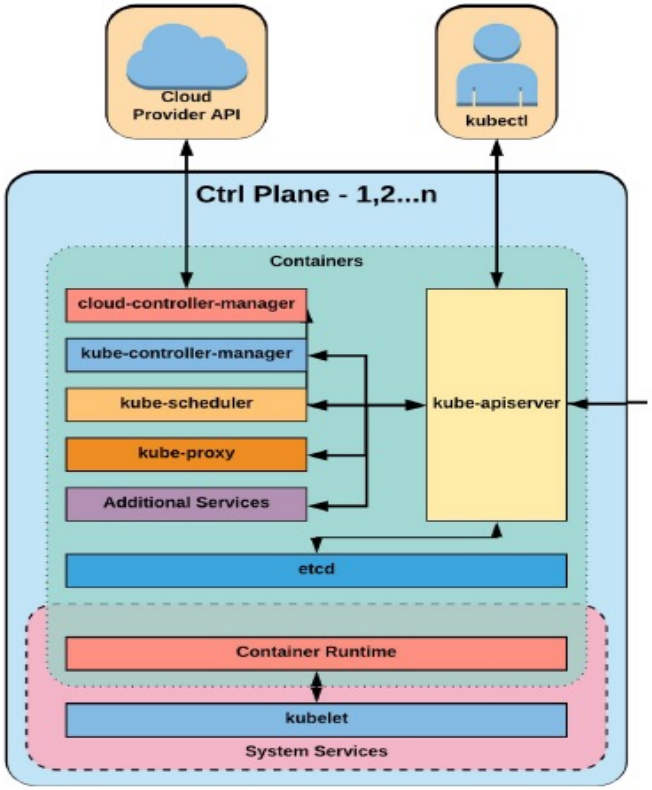
L’architettura di Kubernetes



Control Plane

Multiple components that can run on a single master node or be split across multiple (master) nodes and replicated to ensure high availability:

- **API Server**: communication center for developers, sysadmin and other Kubernetes components
- **Scheduler**: assigns a worker node to each deployable component
- **Controller Manager**: performs cluster-level functions (replication, keeping track of worker nodes, handling nodes failures…)
- **etcd**: reliable distributed data store where the cluster configuration is persisted



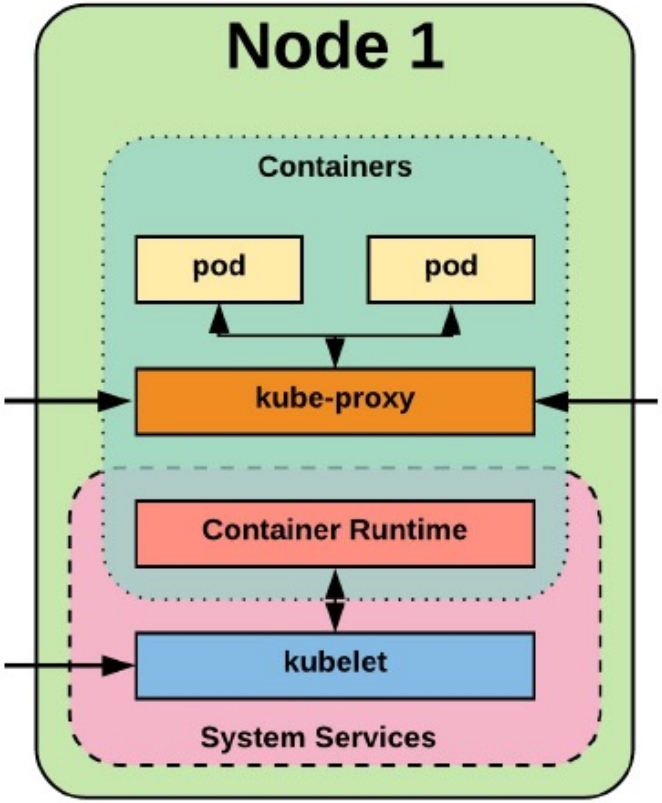
Worker Node

Machines that run containerized applications. It runs, monitors and provides services to applications via components:

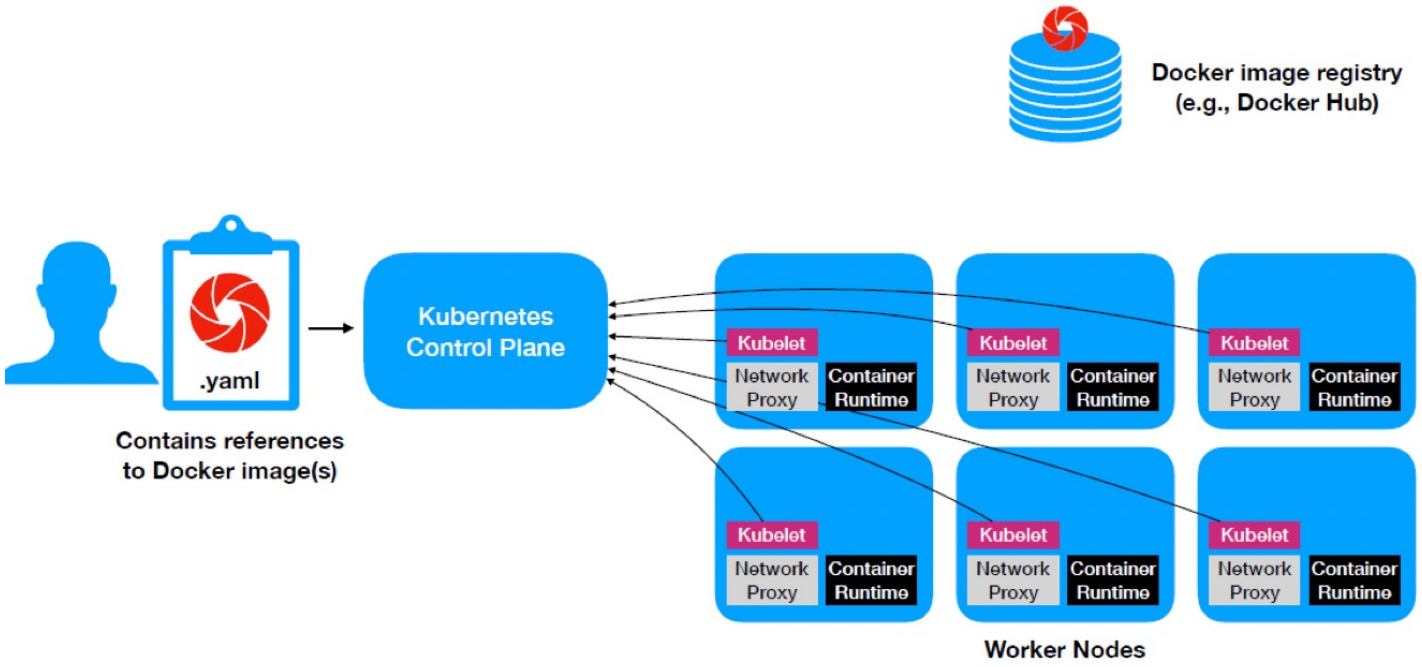
Docker, rkt, or another container runtime: runs the containers

Kubelet: talks to API server and manages containers on its node

Network Proxy: load balance network traffic between application components



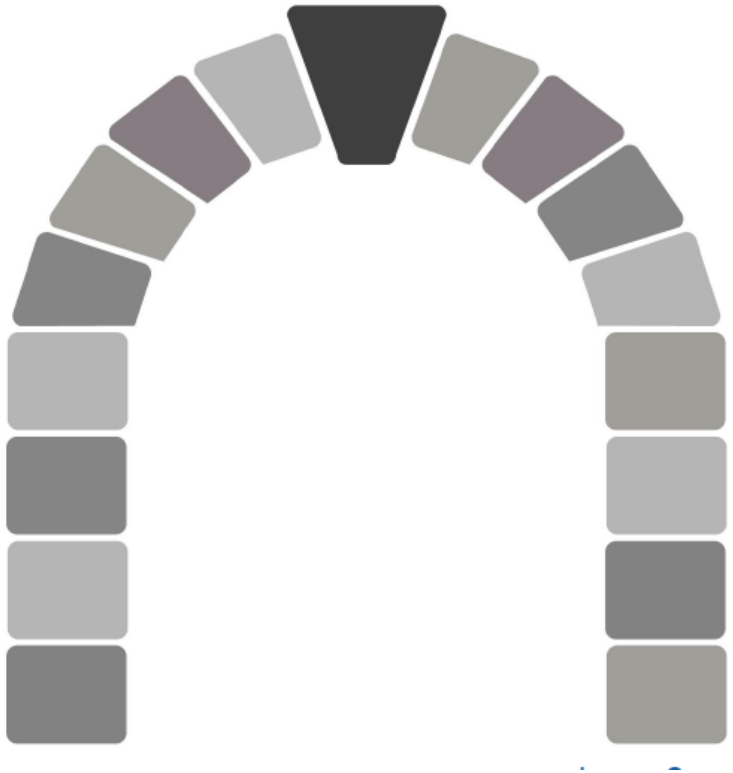
Eseguire un'applicazione su Kubernetes



Kubernetes APIs

The **REST API** is the true **keystone** of Kubernetes.

Everything within the Kubernetes platform is treated as an **API Object** and has a corresponding entry in the API itself.



Kubernetes APIs

- Objects within Kubernetes are a "*record of intent*"
  - Persistent entity that represent the desired state of the object within the cluster.

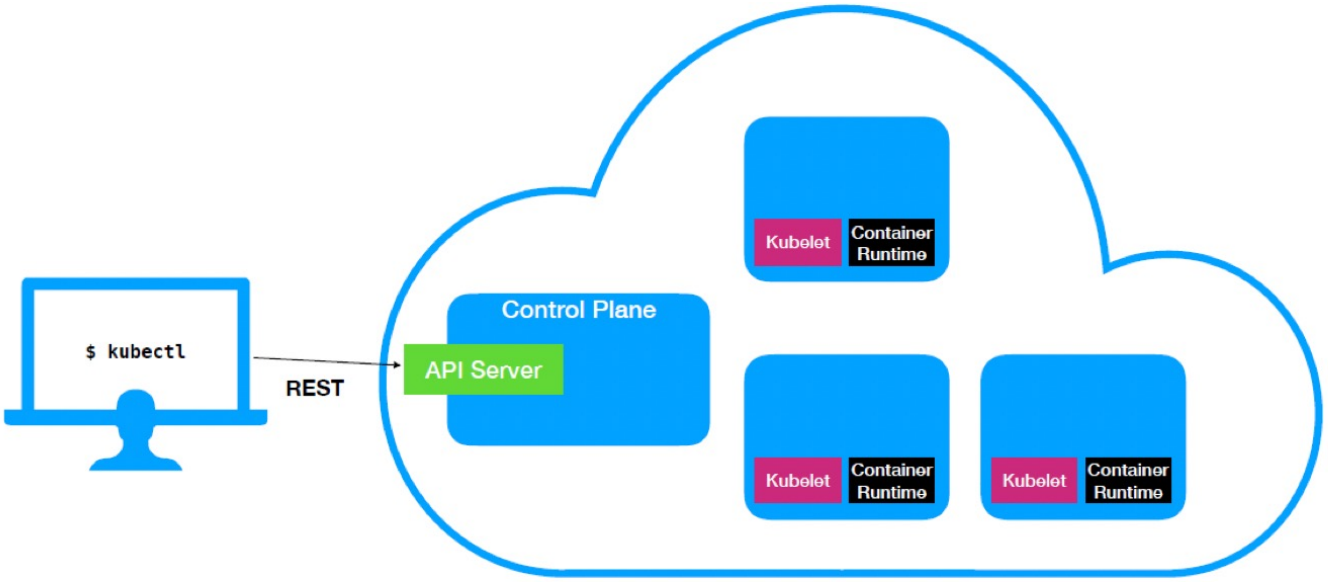
- At a minimum all objects **MUST** have an `apiVersion`, `kind`, and poses the nested fields `metadata.name`, `metadata.namespace`, and `metadata.uid`.

Kubernetes APIs

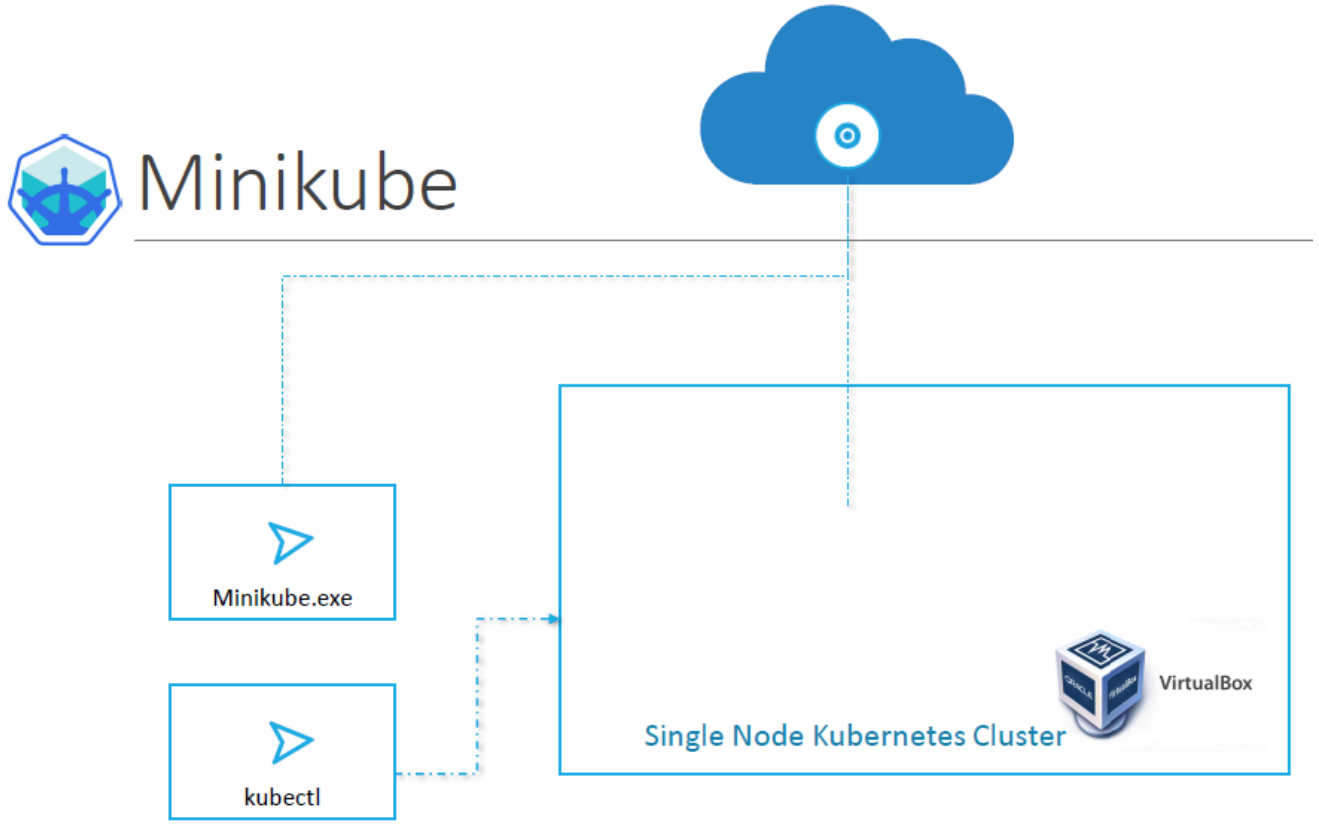
- `apiVersion:` Kubernetes API version of the Object
- `kind:` Type of Kubernetes Object
- `metadata.name:` Unique name of the Object
- `metadata.namespace:` Scoped environment name that the object belongs to (will default to current).
- `metadata.uid:` The (generated) uid for an object.

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-example
  namespace: default
  uid: f8798d82-1185-11e8-94ce-080027b3c7a6
```

Kubernetes APIs

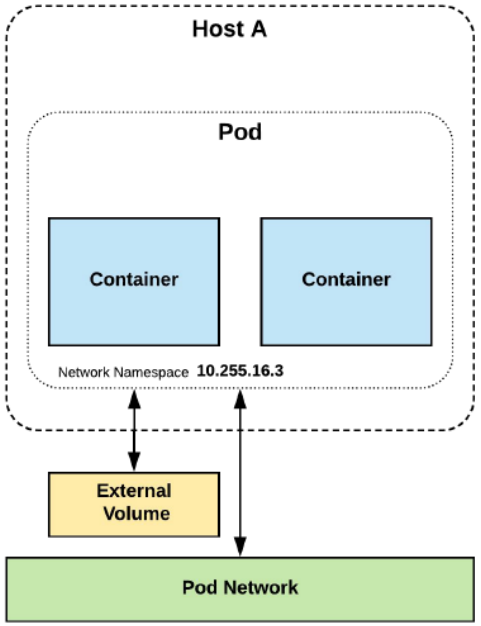


Minikube


Minikube
Minikube.exe
kubectl
VirtualBox
Single Node Kubernetes Cluster

Pods

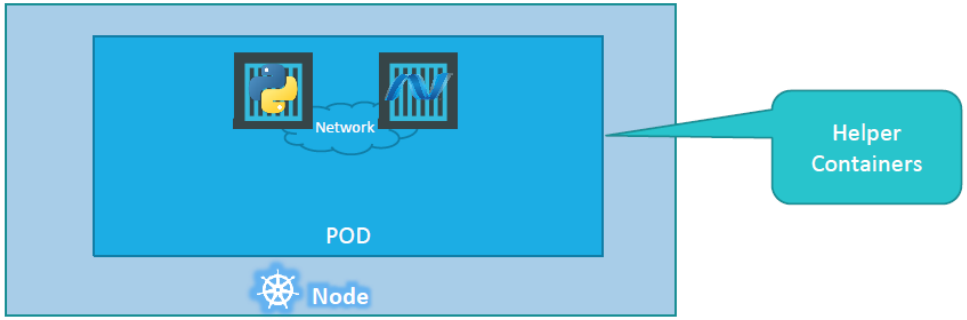
- A pod is the **atomic unit** of Kubernetes.
- It is the foundational building block of Kubernetes Workloads.
- Pods are one or more containers that share volumes, a network namespace, and are a part of a **single context**.



Multi container Pods

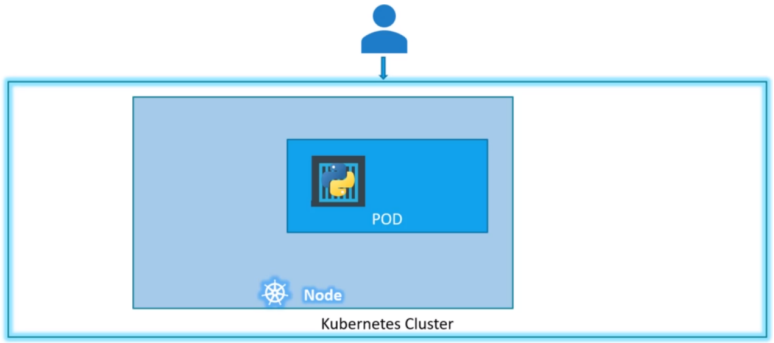
E’ possibile inserire più di un Container all’interno del Pod.

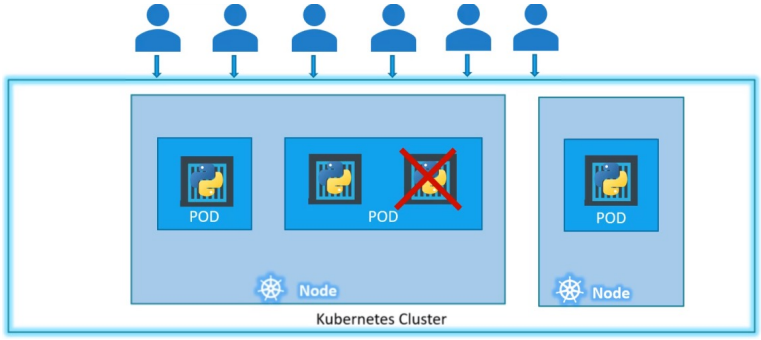
Questa pratica è utile per avere uno o più Containers che supportano il Container principale (Sidecar pattern)



Pods replicas

Quando il carico utente aumenta, la pratica comune è quella di aumentare il fattore di replica dei Pod (le istanze) e non il numero di container in un singolo Pod





Running the first Pod

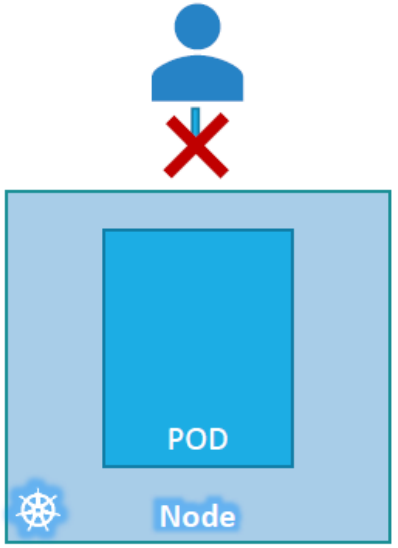
kubectl

```
kubectl run nginx --image nginx
```

```
kubectl get pods
```

```
C:\Kubernetes>kubectl get pods
NAME                    READY     STATUS              RESTARTS   AGE
nginx-8586cf59-whssr    0/1       ContainerCreating   0          3s
```

```
C:\Kubernetes>kubectl get pods
NAME                    READY     STATUS    RESTARTS   AGE
nginx-8586cf59-whssr    1/1       Running   0          8s
```



Running the first Pod

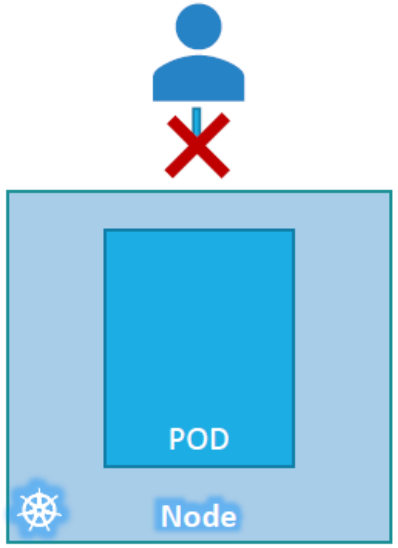
kubectl

```
kubectl run nginx --image nginx
```

```
kubectl get pods
```

```
C:\Kubernetes>kubectl get pods
NAME                    READY     STATUS              RESTARTS   AGE
nginx-8586cf59-whssr    0/1       ContainerCreating   0          3s
```

```
C:\Kubernetes>kubectl get pods
NAME                    READY     STATUS    RESTARTS   AGE
nginx-8586cf59-whssr    1/1       Running   0          8s
```



Pods con YAML

Ogni risorsa Kubernetes può esser descritta in YAML, la cui struttura di base è formata dalle seguenti chiavi (tutte obbligatorie)

pod-definition.yml

```
apiVersion:
kind:
metadata:



spec:
```

Pods con YAML

apiVersion rappresenta la versione delle API che useremo per creare la nostra risorsa (in questo caso un Pod). E' necessario specificare l'apiVersion corretta per ogni risorsa (vedi tabella).

pod-definition.yml

```
apiVersion: v1
kind:
metadata:



spec:
```

Kind	Version
POD	v1
Service	v1
ReplicaSet	apps/v1
Deployment	apps/v1

Pods con YAML

kind rappresenta il tipo di risorsa da creare. In questo caso il Pod

pod-definition.yml

```yaml
apiVersion: v1
kind: Pod
metadata:



spec:
```

Kind	Version
POD	v1
Service	v1
ReplicaSet	apps/v1
Deployment	apps/v1

Pods con YAML

metadata racchiude delle informazioni fondamentali per il Pod. Qui vediamo il nome (obbligatorio) ed una label (app). Parleremo delle label in seguito.

pod-definition.yml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
      app: myapp

spec:
```

Pods con YAML

spec (specification) racchiude i containers che il Pod avrà al suo interno. La struttura di spec dipende dal tipo di risorsa (in questo caso Pod).

pod-definition.yml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
    type: front-end
spec:
  containers:
    - name: nginx-container
      image: nginx
```

List/Array

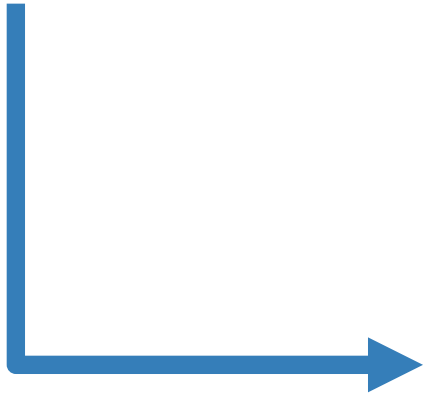
pod-definition.yml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
    type: front-end
spec:
  containers:
    - name: nginx-container
      image: nginx
```

1st Item in List

Pods con YAML

```
$ kubectl create –f pod-definition.yml
```



```
> kubectl get pods
NAME          READY     STATUS    RESTARTS   AGE
myapp-pod     1/1       Running   0          20s
```

```
> kubectl describe pod myapp-pod
Name:         myapp-pod
Namespace:    default
Node:         minikube/192.168.99.100
Start Time:   Sat, 03 Mar 2018 14:26:14 +0800
Labels:       app=myapp
              name=myapp-pod
Annotations:  <none>
Status:       Running
IP:           172.17.0.24
Containers:
  nginx:
    Container ID:   docker://830bb56c8c42a86b4bb70e9c1488fae1bc38663e4918b6c2f5a783e7688b8c9d
    Image:          nginx
    Image ID:       docker-pullable://nginx@sha256:4771d09578c7c6a65299e110b3ee1c0a2592f5ea2618d23e4ffe7a4cab1ce5de
    Port:           <none>
    State:          Running
      Started:      Sat, 03 Mar 2018 14:26:21 +0800
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from default-token-x95w7 (ro)
Conditions:
  Type           Status
  Initialized    True
  Ready          True
  PodScheduled   True
Events:
  Type    Reason                 Age   From               Message
  ----    ------                 ----  ----               -------
  Normal  Scheduled              34s   default-scheduler  Successfully assigned myapp-pod to minikube
  Normal  SuccessfulMountVolume  33s   kubelet, minikube  MountVolume.SetUp succeeded for volume "default-token-x95w7"
  Normal  Pulling                33s   kubelet, minikube  pulling image "nginx"
  Normal  Pulled                 27s   kubelet, minikube  Successfully pulled image "nginx"
  Normal  Created                27s   kubelet, minikube  Created container
```

Altri Pods con YAML

Single container

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-example
spec:
  containers:
  - name: nginx
    image: nginx:stable-alpine
    ports:
    - containerPort: 80
```

Multi container

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: multi-container-example
spec:
  containers:
  - name: nginx
    image: nginx:stable-alpine
    volumeMounts:
    - name: html
      mountPath: /usr/share/nginx/html
  - name: content
    image: alpine:latest
    command: ["/bin/sh", "-c"]
    args:
      - while true; do
          date >> /html/index.html;
          sleep 5;
        done
    volumeMounts:
    - name: html
      mountPath: /html
  volumes:
  - name: html
    emptyDir: {}
```

LAB

https://github.com/sunnyvale-academy/ITS-ICT_Containers

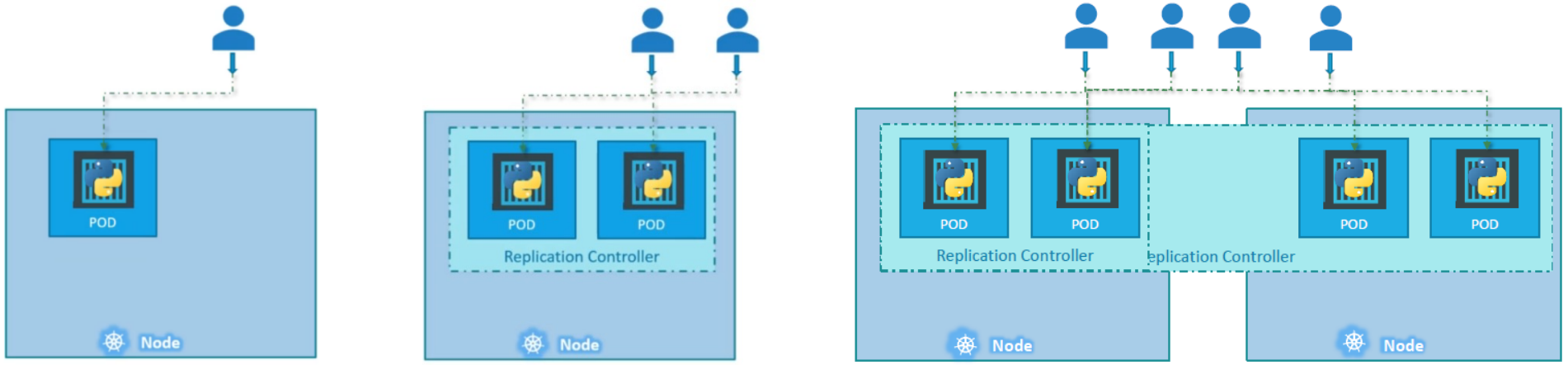
Lab 19 – Pod

ReplicationController, ReplicaSet e Deployment

Pod lifecycle (Pod status)

Value	Description
`Pending`	The Pod has been accepted by the Kubernetes system, but one or more of the Container images has not been created. This includes time before being scheduled as well as time spent downloading images over the network, which could take a while.
`Running`	The Pod has been bound to a node, and all of the Containers have been created. At least one Container is still running, or is in the process of starting or restarting.
`Succeeded`	All Containers in the Pod have terminated in success, and will not be restarted.
`Failed`	All Containers in the Pod have terminated, and at least one Container has terminated in failure. That is, the Container either exited with non-zero status or was terminated by the system.
`Unknown`	For some reason the state of the Pod could not be obtained, typically due to an error in communicating with the host of the Pod.
`Completed`	The pod has run to completion as there's nothing to keep it running eg. Completed Jobs.
`CrashLoopBackOff`	This means that one of the containers in the pod has exited unexpectedly, and perhaps with a non-zero error code even after restarting due to restart policy.

ReplicationController (deprecato in favore di ReplicaSet)



ReplicationController

rc-definition.yml

```yaml
apiVersion: v1
kind: ReplicationController
metadata:
  name: myapp-rc
  labels:
      app: myapp
      type: front-end
spec:
  template:
```

POD

pod-definition.yml

```yaml
apiVersion: v1
kind: Pod

metadata:
  name: myapp-pod
  labels:
     app: myapp
     type: front-end
spec:
  containers:
  - name: nginx-container
    image: nginx
```

ReplicationController

rc-definition.yml

```yaml
apiVersion: v1
kind: ReplicationController
metadata:
  name: myapp-rc
  labels:
      app: myapp
      type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front
    spec:
      containers:
      - name: nginx-container
        image: nginx
```

POD

ReplicationController

Per specificare il numero di repliche aggiungiamo:

```
replicas: 3
```

ReplicationController

rc-definition.yml

```yaml
apiVersion: v1
kind: ReplicationController
metadata:
  name: myapp-rc
  labels:
      app: myapp
      type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      containers:
      - name: nginx-container
        image: nginx

  replicas: 3
```

```
$ kubectl create -f rc-definition.yml
```

```
$ kubectl get replicationcontroller
```

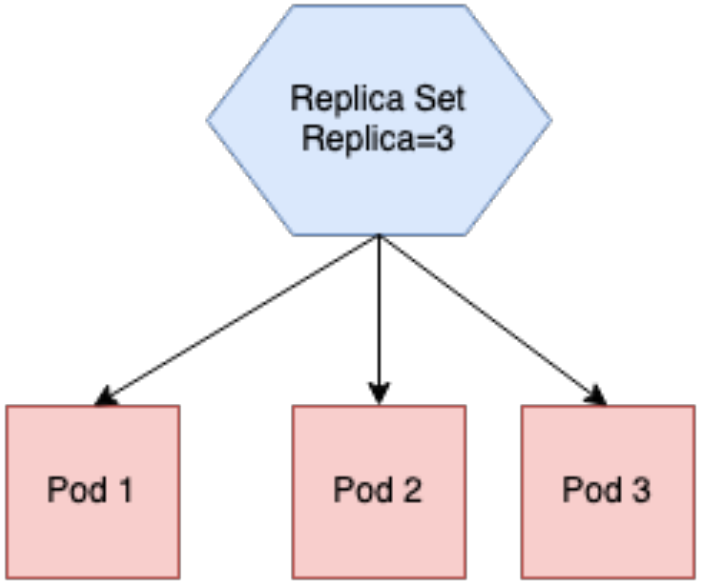
```
$ kubectl get pods
```

LAB

https://github.com/sunnyvale-academy/ITS-ICT_Containers

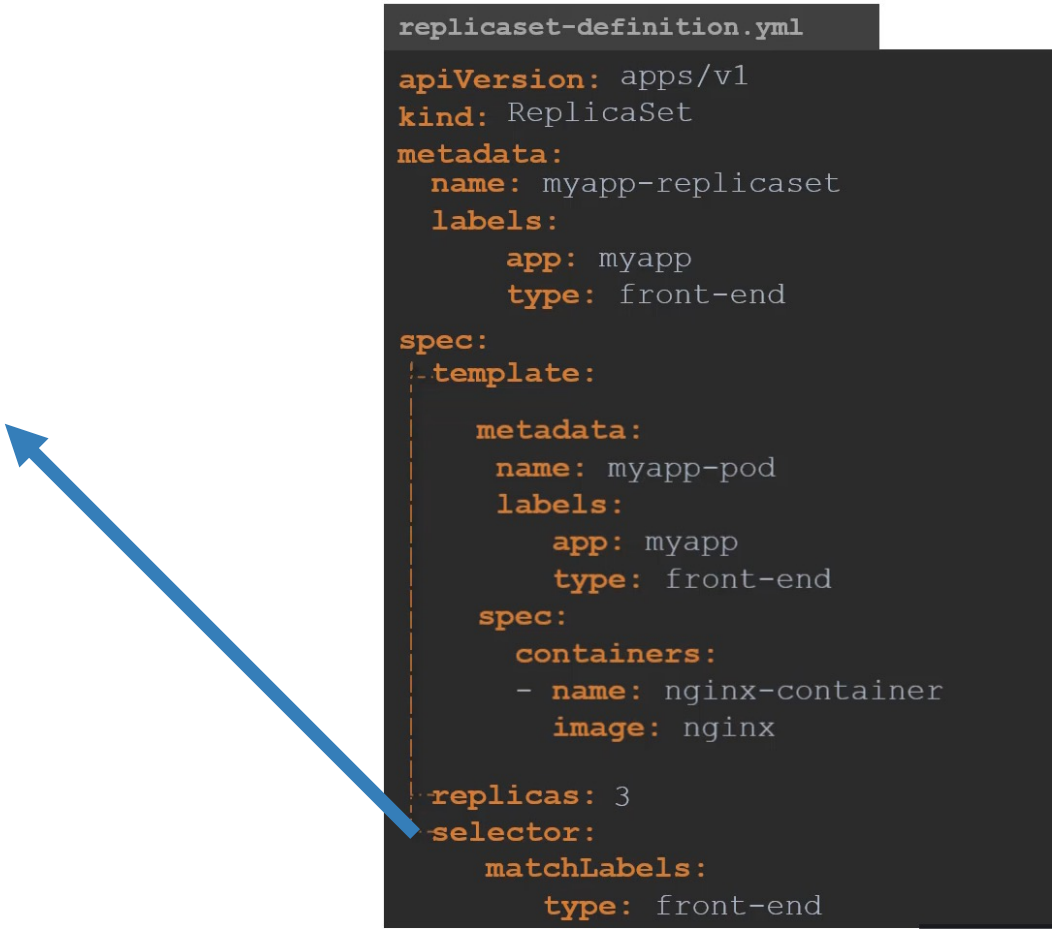
Lab 20 – ReplicationController

ReplicaSet



ReplicaSet

Con il Selector, ReplicaSet può anche gestire Pod che non ha creato



replicaset-definition.yml

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp-replicaset
  labels:
      app: myapp
      type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      containers:
      - name: nginx-container
        image: nginx

  replicas: 3
  selector:
    matchLabels:
      type: front-end
```

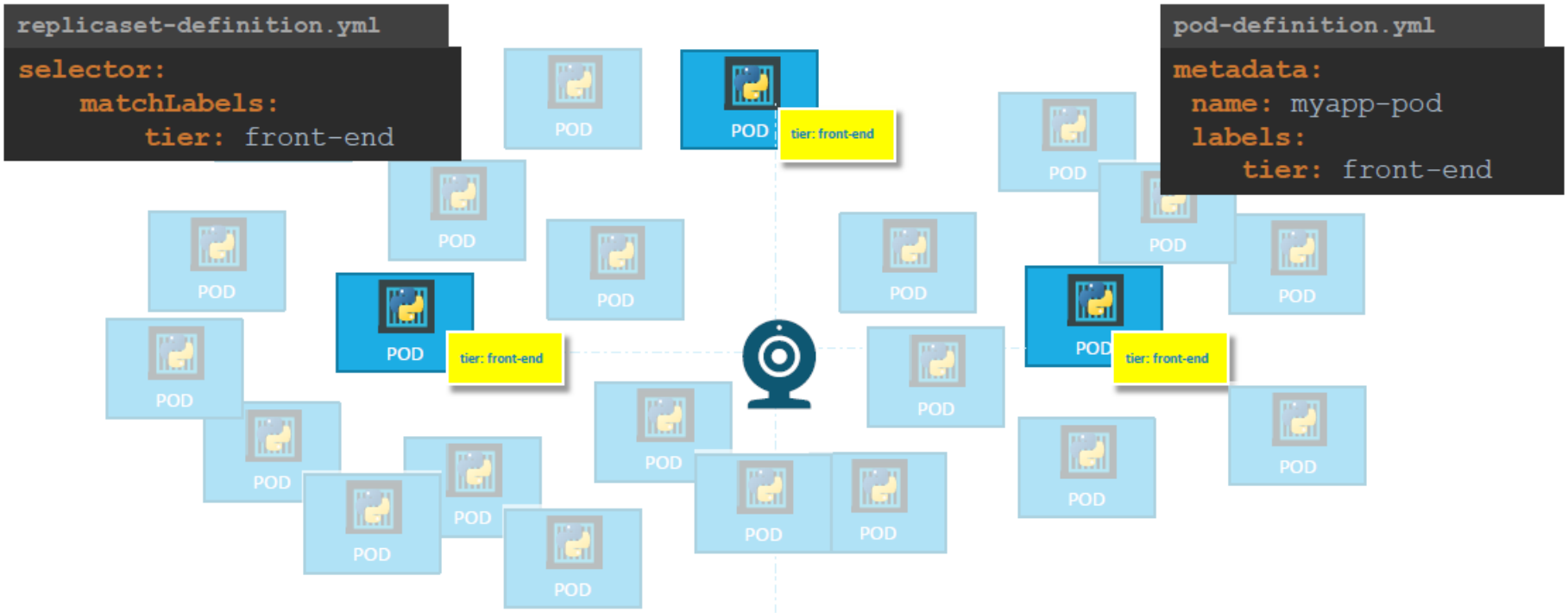
ReplicaSet

```
$ kubectl create –f replicaset-definition-definition.yml
```

```
$ kubectl get replicaset
```

```
$ kubectl get pods
```

Labels and Selectors



Labels and Selectors

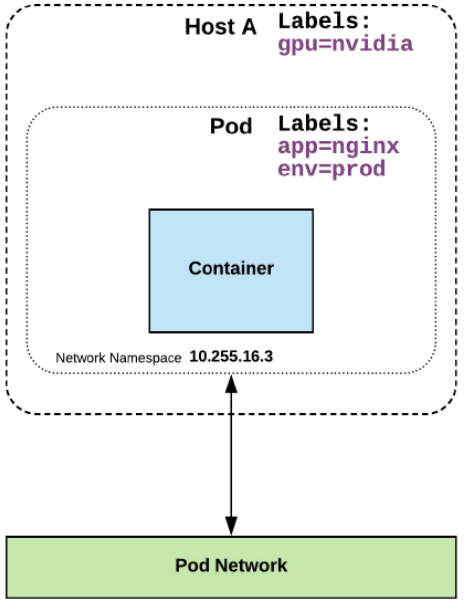
replicaset-definition.yml

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp-replicaset
  labels:
      app: myapp
      type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
        tier: front-end
    spec:
      containers:
      - name: nginx-container
        image: nginx
  replicas: 3
  selector:
    matchLabels:
      tier : front-end
```

Template

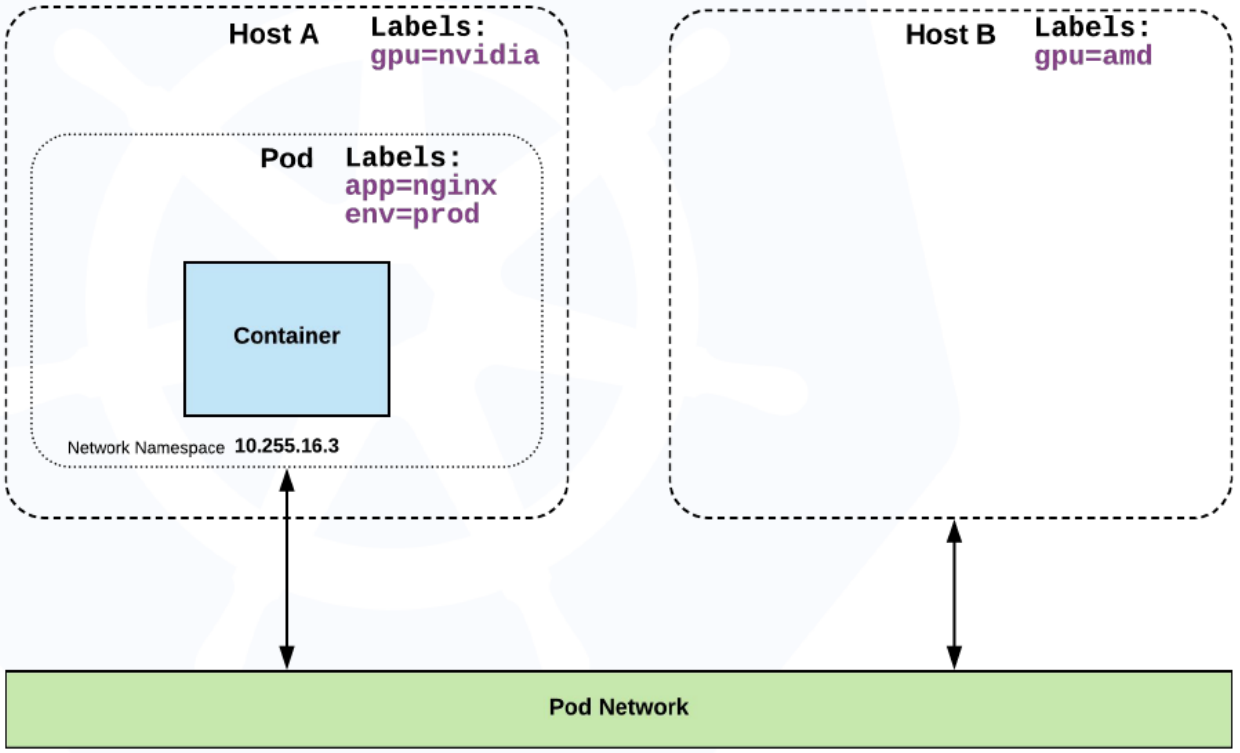
Labels and Selectors

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-label-example
  labels:
    app: nginx
    env: prod
spec:
  containers:
  - name: nginx
    image: nginx:stable-alpine
    ports:
    - containerPort: 80
```



Labels and Selectors

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-label-example
  labels:
    app: nginx
    env: prod
spec:
  containers:
  - name: nginx
    image: nginx:stable-alpine
    ports:
    - containerPort: 80
  nodeSelector:
    gpu: nvidia
```



Labels and Selectors

Equality based selectors allow for simple filtering (=,==, or !=).

```
selector:
  matchLabels:
    gpu: nvidia
```

Set-based selectors are supported on a limited subset of objects. However, they provide a method of filtering on a set of values, and supports multiple operators including: `in`, `notin`, and `exist`.

```
selector:
  matchExpressions:
    - key: gpu
      operator: in
      values: [“nvidia”]
```

Scaling

replicaset-definition.yml

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp-replicaset
  labels:
      app: myapp
      type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
         app: myapp
         type: front-end
    spec:
      containers:
      - name: nginx-container
        image: nginx

  replicas: 6
  selector:
    matchLabels:
       type: front-end
```

```
$ kubectl replace -f
replicaset-definition.yml
```

Scaling

```
$ kubectl scale –replicas=6 replicaset-definition.yml
```

```
$ kubectl scale –replicas=6 replicaset myapp-replicaset
```

LAB

https://github.com/sunnyvale-academy/ITS-ICT_Containers

Lab 21 – ReplicaSet